ISLAMIC UNIVERSITY OF TECHNOLOGY

Orthogonal Defect Classification on Defects4J

Using an LLM-Driven Scientific Approach

Thesis Pre-Defense Presentation

Presented by

Hasin Mahtab Alvee (210042174)
Mohammad Nahiyan Kabir (210042168)
Md. Sakib Hossain (210042133)

Supervised by

Lutfun Nahar Lota, Assistant Professor | Ishmam Tashdeed, Lecturer

Network and Data Analysis Group (NDAG)

Presentation Outline

1 **Introduction**
Defects4J & Bug Categorization

2 **Literature Review**
Existing Classifications & LLM Approaches

3 **Problem Statement**
Research Gap & Motivation

4 **Objectives**
Aims of the Study

5 **Proposed Methodology**
Pipeline Architecture & Design

6 **Classification Findings**
Classification & Comparison Data

7 **Discussion & Analysis**
Divergence Patterns & Case Study

8 **Limitations**
Known Constraints

9 **Future Work**
Roadmap & Extensions

10 **Conclusion**
Summary & Closing

Introduction to Defects4J

What is Defects4J?

- Curated database of real, reproducible Java bugs with developer written patches
- Provides per-bug artifacts: bug reports, triggering tests, buggy code, fixed code
- Standard benchmark for empirical SE research (fault localization, APR, testing)
- Grown to 864+ active bugs across 17 Java projects [3]
- Includes CLI tools for checkout, compilation, testing, and coverage analysis

Continued Relevance

- Actively used in 2024 fault localization studies [3]
- Standard benchmark for large-scale APR experiments [5]
- Manual inspections confirm diagnostic value of test-suite artifacts [6]

Key Artifacts per Bug

Bug Report
Original issue description

Triggering Tests
Tests that expose the bug

Buggy Source Code
Pre-fix Java source

Fixed Source Code
Developer-patched code

Code Diffs
Buggy-to-Fixed changes

CLI Tools
Checkout, compile, test, coverage

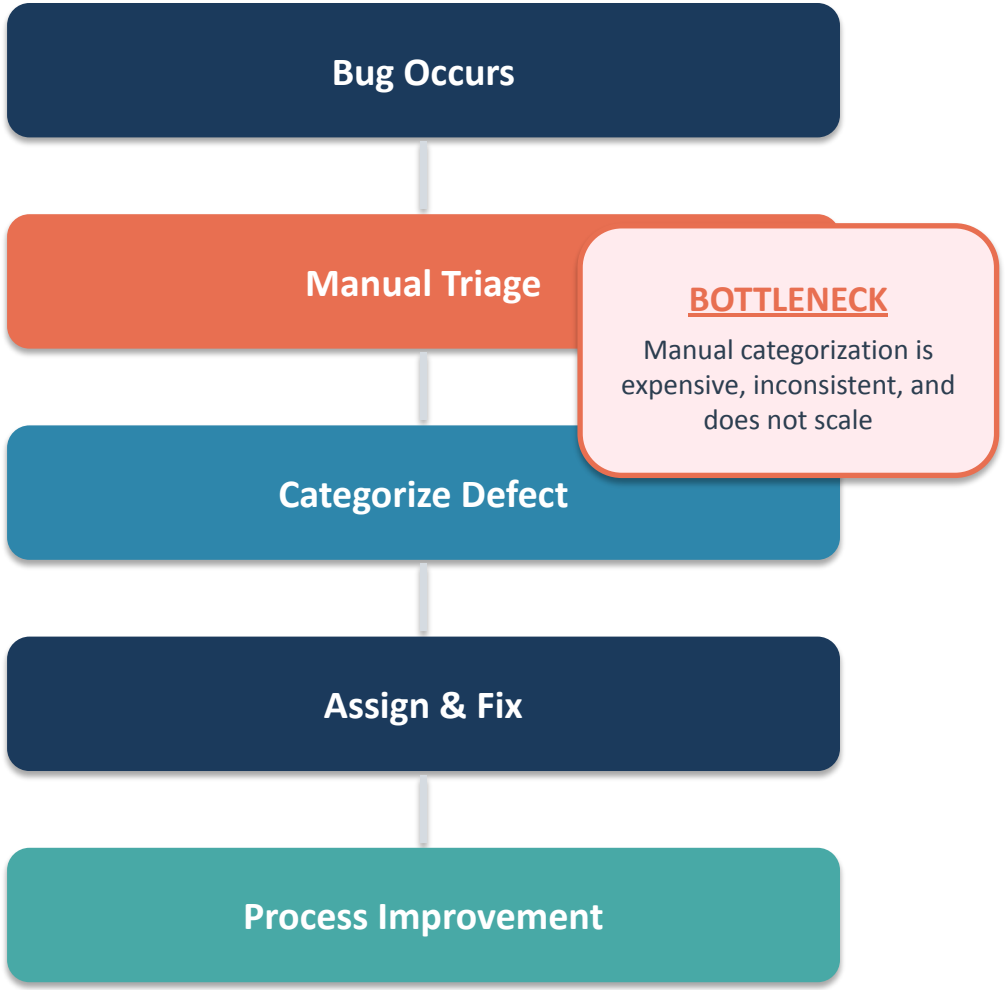
Why Bug Categorization Matters in the SDLC

The Need for Structured Categorization

- Different bug types behave differently and require different fix strategies [7]
- Categorization is operationally necessary for optimizing debugging [12]
- Automated classification improves triage and maintenance efficiency [8]
- Predicting fault category from bug reports helps prioritize early [9]
- Structured categorization provides actionable in-process measurements [12]

Current Challenge

- Manual labeling is the gold standard but too expensive at scale [8]



Existing Defects4J Classification Attempts

Structural / Graph-Based

Van der Spuy & Fischer (2025) [4]

- CFG/DFG-based classification
- 488 bugs across 7 projects
- 6 control-flow + 2 data-flow classes
- Captures HOW bugs manifest syntactically

Patch-Level Taxonomy

Sobreira et al. (2018) [2]

- Repair-action classification
- 395 bugs from D4J v1.2 only
- 43% involve conditional changes
- Manual patch taxonomy extraction

Small-Sample Inspection

J. Jiang et al. (2017) [6]

- Random sample of 50 bugs
- Identifies fixability patterns
- Manual analysis reveals detail
- Self-acknowledged validity threat

None of these approaches apply a semantic, root-cause taxonomy to the full Defects4J dataset. All are either structural-only, patch-level, or limited to small subsets.

Limitations of Existing Classification Approaches

Critical Limitations in Current Defects4J Classification

✗ Coverage Gap

No study has fully labeled all 864+ D4J bugs; existing work covers only 50–488 bugs [2][4][6]

✗ Structural-Only Taxonomy

CFG/DFG approaches capture ***HOW*** bugs manifest, not ***WHY*** they exist ***semantically*** [4]

✗ Patch-Level, Not Root-Cause

Repair-action classification describes fix patterns, not the ***underlying mechanism*** [2]

✗ Post-Fix Dependency

Existing ML approaches require labeled training data or post-fix code [14]

✗ Descriptive, Not Diagnostic

Current labels are too low-level or inconsistent for decision making in triage & QA [7]

LLM-Based Defect Classification: A Viable Path

Evidence Supporting LLM-Driven Classification

✦ Issue Report Classification [10]

LLMs effectively classify issue reports — validates the LLM-for-defect-classification hypothesis

✦ Fine-Grained Categorization [11]

LLMs + prompt engineering + few-shot achieve fine-grained bug report categorization

✦ Chain-of-Thought Prompting [19]

CoT prompting improves LLM classification accuracy by >500% on structured tasks

✦ ODC-Aligned Root Cause Analysis [18]

LLMs generate structured, ODC-aligned root cause explanations from bug reports & stack traces

LLMs vs Traditional ML

Traditional ML (SVM/NB)

- ~77–82% accuracy [13][14]
- Requires labeled training data
- Sparse text limitations

LLM-Based Approach

- Code-aware reasoning
- Zero-shot / few-shot capable
- Structured taxonomy guidance

The Taxonomy Problem for LLMs

LLMs alone are not enough — without a structured taxonomy, classification collapses

✖ Hallucination Without Structure

LLMs hallucinate heavily when classifying bugs without concrete stack traces or a guiding taxonomy [18]

✖ Inconsistent Labels

Without standardization, different LLM runs produce different category labels for the same bug [11]

✖ Incompatible Taxonomies

Ad hoc ML classification yields a proliferation of incompatible categories that can't be compared [8]

✖ Unactionable Results

Classification without a principled framework produces vague labels with no operational value [14]

Research Gap

The Core Gap

No study has fully labeled all Defects4J bugs. Existing work covers only 50–488 bugs, uses structural-only taxonomies, or requires post-fix code — preventing pre-triage use. There exists no context-aware, semantically-grounded, scalable classification of the full Defects4J dataset.

What's Needed: A pipeline that is →

✓ Context-Aware

Multi-modal evidence: code, tests, bug reports, stack traces

✓ Semantic

Root-cause classification, not syntactic patterns

✓ Taxonomy-Constrained

Taxonomy prevents hallucination & ensures comparability

✓ Scalable

LLM-based for full dataset without manual labeling

Orthogonal Defect Classification (ODC)

OPENER Attributes (When Defect Found)

- Activity: Development phase when defect found
- Trigger: Testing condition that exposed defect
- Impact: Effect on customer / system

CLOSER Attributes (When Defect Fixed)

- Defect Type: Root cause category (7 types)
- Target: Design/Code artifact affected
- Qualifier: Missing, Incorrect, or Extraneous
- Age: Base, New, or Rewritten code
- Source: In-house, Library, or Vendor

The 7 ODC Defect Types (Target = Design/Code)

- Algorithm/Method — Wrong computational logic [C&DF]
- Assignment/Initialization — Wrong value / initialization
- Checking — Missing / wrong validation
- Timing/Serialization — Concurrency / ordering issue
- Function/Class/Object — Design-level gap [Structural]
- Interface/O-O Messages — Component boundary issue
- Relationship — Entity association problem

ODC × Defects4J: Natural Artifact Mapping

OPENER (Discovery Story)

Tests → **Activity**

Test type tells us WHAT was being done when defect surfaced

Stack Traces → **Trigger**

Runtime context shows WHAT condition woke the dormant defect

Bug Reports → **Impact**

User-visible symptom describes the consequence (crash, wrong output)

CLOSER (Repair Story)

Code Diffs → **Type + Qualifier**

Diff reveals the semantic mistake (Type) and how it manifested (Missing / Incorrect / Extraneous)

Modified Class → **Target + Age**

WHERE the fix was applied (Target = Code) and WHEN the defect was injected (Base / New)

Research Objectives

RO 1

Build an automated pipeline that classifies Defects4J bugs into ODC defect types using LLM reasoning guided by structured scientific debugging.

RO 2

Collect bug evidence in two modes — pre-fix (symptoms only) and post-fix (with the actual code fix) — mapping Defects4J artifacts to ODC opener and closer attributes.

RO 3

Apply Zeller's scientific debugging methodology as an LLM prompting strategy to produce explainable, step-by-step classifications.

RO 4

Design a 4-tier evaluation framework that compares pre-fix vs post-fix classifications using strict match, top-2 match, family match, and Cohen's Kappa.

Pipeline Architecture Overview

1. COLLECT

Evidence Gathering

D4J CLI ⟶ multi-modal evidence ⟶ context.json

→

2. CLASSIFY

LLM Classification

context.json ⟶ ODC prompt ⟶ LLM ⟶ classification.json

→

3. COMPARE

Evaluation Framework

Pre-fix vs Post-fix ⟶ multi-tier analysis

Key Architectural Principles

- File-oriented, synchronous — no database, queue, or async orchestration
- Multi-provider LLM support (Gemini, OpenRouter, OpenAI-compatible)
- Pre-fix evidence by default; post-fix via explicit --include-fix-diff
- Hidden oracle exclusion: classes.modified never enters the LLM prompt
- Inspired by AutoSD [15]: Zeller's scientific debugging in structured prompts [16]

Evidence Collection: Building context.json

1. **Query D4J Metadata**
   Project, bug info, report URL, modified classes

2. **Fetch Bug Report**
   JIRA / GitHub API / HTML → opener-side evidence

3. **Checkout & Compile**
   Buggy version checkout → compile → test

4. **Parse Failures**
   Extract test names, stack traces, exception headlines

5. **Select Suspicious Frames**
   Filter framework / JDK frames → project source priority

6. **Extract Code Snippets**
   Production (±12 lines) + test source (±18 lines)

7. **Coverage (optional)**
   Cobertura XML → line/branch execution rates

8. **Serialize context.json**
   Complete evidence payload for classification

Evidence Modes: Pre-fix vs Post-fix

PRE-FIX MODE (Default)

LLM Sees
Symptoms only

Evidence
Stack traces, tests, errors,
code snippets, bug report

Perspective
"What went wrong?"

ODC Side
Opener (trigger, activity, impact)

≈ A doctor diagnosing from symptoms

POST-FIX MODE (--include-fix-diff)

LLM Sees
Symptoms + actual code fix

Evidence
All pre-fix evidence +
buggy→fixed diff

Perspective
"What was changed?"

ODC Side
Closer (type, qualifier, age, source)

≈ A pathologist with biopsy results

ODC's opener/closer model requires evidence from both perspectives [12].
Pre-fix captures opener evidence, post-fix captures closer evidence.

ODC Taxonomy Integration & Prompt Design

System Prompt: 6 Components

1. ODC Taxonomy

7 types with definitions, indicators & contrastive "distinguish_from" guidance

2. Anti-Bias Rules

"Do NOT default to Function/Class/Object" — simpler explanations first

3. Scientific Debugging

Hypothesize → Observe → Predict → Examine → Conclude [15][16]

4. Diagnostic Tree

7 questions, each maps to one ODC type (FCO deliberately last)

5. JSON Schema

Structured output: reasoning, confidence, alternatives, ODC fields

6. Few-Shot (×5)

Contrastive examples: Symptom → Code → Type → NOT X [11][19]

Scientific Classification Protocol

Step 1: HYPOTHESIZE → **Step 2: OBSERVE** → **Step 3: PREDICT** → **Step 4: EXAMINE** → **Step 5: CONCLUDE**

Step 1: HYPOTHESIZE	Step 2: OBSERVE	Step 3: PREDICT	Step 4: EXAMINE	Step 5: CONCLUDE
Form a root-cause hypothesis from the bug report	Examine failure symptoms: error messages, stack traces	What code pattern would explain this defect type?	Check code snippets against the prediction	Select ODC type for the root cause mechanism

7 Question Diagnostic Decision Tree

Q1: Missing guard/condition?
→ Checking

Q2: Wrong value or init?
→ Assignment/Init

Q3: Wrong computation logic?
→ Algorithm/Method

Q4: Component boundary issue?
→ Interface/O-O

Q5: Execution order dependent?
→ Timing/Serial

Q6: Entity association issue?
→ Relationship

Q7: Needs design-level fix?
→ Fun/Cls/Obj*

Classification Pipeline & Validation

Prompt Construction

context.json → **System Prompt** → **User Prompt** → **LLM API Call**

- context.json: Load bug evidence
- System Prompt: ODC taxonomy + scientific debug + few-shot
- User Prompt: Evidence payload
- LLM API Call: Gemini / OpenRouter

Post-Processing & Validation

Extract JSON → **Validate Type** → **Canonicalize** → **Write Outputs**

- Extract JSON: Parse from raw LLM response
- Validate Type: Check against 7 canonical ODC names
- Canonicalize: Overwrite family + normalize fields
- Write Outputs: classification.json + report.md

Evaluation Strategy & Framework

Defects4J lacks complete ODC ground-truth labels, so predicted classifications cannot be reliably validated. Therefore, precision and recall cannot be meaningfully computed without accurate reference annotations. To address this, we adopt a tier-based evaluation approach with progressively relaxed agreement criteria.

Tier 1

Strict Match

Exact odc_type agreement

Expected: 40–60%

Tier 2

Top-2 Match

Primary or alternative overlap

Expected: 60–80%

Tier 3

Family Match

Same ODC family (C&DF / Structural)

Expected: 70–90%

Tier 4

Cohen's Kappa

Statistical measure of agreement beyond chance

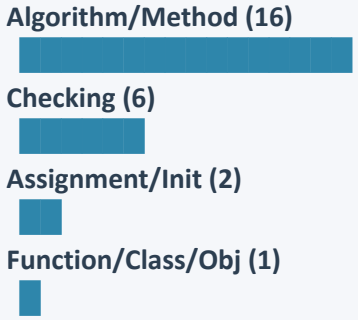
Expected: $\kappa \geq 0.50$

Even human ODC experts disagree on 20–30% of bugs [12]. A tiered framework captures both exact agreement and meaningful near-agreement, matching industry evaluation practice [14][22].

Classification Findings

34	17	73.5%	94.1%	97.1%
Bug Pairs Analyzed	Defects4J Projects	Strict Type Match	Top-2 Match	Family Match

Type Agreement Distribution (25/34 stable)



Type Shift Patterns (9/34 changed)

Checking → Algorithm/Method [3]
Algorithm/Method → Checking [2]
Checking → Assignment/Init [1]
Checking → Interface/O-O Msg [1]
Algorithm/Method → Assignment/Init [1]
Function/Class/Obj → Relationship [1]

Out of 9 bugs where the type shifted, 7 still had overlap through alternative types. Only 2 bugs (JacksonCore-24, Math-34) had no alternative overlap — but both matched at the family level.

Discussion & Analysis

25 (73.5%)
Exact Match
Same type in both modes

7 (20.6%)
Soft Divergence
Type shifted, cross-alt match

2 (5.9%)
Hard Divergence
No alternative overlap

Why Do Types Diverge?

Symptom vs Cause
Pre-fix sees symptoms;
post-fix sees the actual fix

ODC Boundary Ambiguity
Checking ↔ Algorithm boundary
is inherently ambiguous

Information Gap
Post-fix has code change;
pre-fix infers from signals

Case Study: Closure-140 (Soft Divergence)

Pre-fix → Checking (validation/guard issue) | Post-fix → Algorithm/Method (procedural correction)

→ **Cross-alternative match: each primary type appears in the other side's alternatives — exactly what ODC theory predicts [12]**

Limitations

L1: No Ground-Truth Labels

Defects4J has no ODC labels from human experts. We evaluate using self-consistency, not a gold standard. [8][12]

L2: LLM Reproducibility

Outputs can vary between models and runs. Temperature=0.0 helps but doesn't fully eliminate variability. [11]

L3: Data Contamination

LLMs may have seen D4J bugs during training, especially popular projects like Lang and Math. [3]

L4: Heuristic Hints

Opener/closer hints are keyword-based, not validated by trained assessors. [12]

L5: Evidence Quality

Not all projects give equally rich evidence — stack trace depth varies between projects.

L6: Java-Only Scope

Evidence collection is tied to D4J's Java ecosystem. Classification logic itself is language-independent.

Future Work

FW 1

Full Dataset Classification

Scale to all 864+ active D4J bugs — create the first complete ODC-labeled Defects4J dataset [9]

FW 2

Multi-Model Consensus

Classify with 3+ models (Gemini, GPT-4, Claude) and use majority voting to reduce variability [10]

FW 3

Validated ODC Opener

Replace keyword heuristics with proper Activity → Trigger mapping using D4J test metadata [12]

FW 4

ODC-Guided APR

Use ODC labels to pick the right repair tool: e.g. Checking → conditional templates [5]

FW 5

Cross-Dataset Extension

Adapt the pipeline for other datasets like Bears, BugSwarm, and non-Java projects [7]

Conclusion

✦ Research Gap Addressed

First study to apply ODC taxonomy to Defects4J using LLM-driven scientific approach

✦ Pipeline Contribution

Fully automated pipeline: evidence collection → structured prompting → explainable classifications

✦ Methodological Innovation

Combined Chillarege's ODC with AutoSD's scientific debugging into a principled LLM prompting strategy

✦ 4-Tier Evaluation

Strict match, top-2 match, family match, and Cohen's Kappa — accounts for the 20–30% human disagreement ceiling [12]

✦ Practical Impact

Pre-fix ODC classification works — type shifts between modes are expected, not failures

References (1/2)

[1] Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. ISSTA 2014, ACM, pp. 437–440.

[2] Sobreira, V., Durieux, T., Madeiral, F., Monperrus, M., & Maia, M. A. (2018). Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J. IEEE SANER 2018, pp. 130–140.

[3] Rafi, M. N., Chen, A. R., Chen, T.-H. P., & Wang, S. (2024). Revisiting Defects4J for Fault Localization in Diverse Development Scenarios. arXiv:2402.13040.

[4] Van der Spuy, A., & Fischer, B. (2025). An Anatomy of 488 Faults from Defects4J Based on the Control- and Data-Flow Graph Representations of Programs. arXiv:2502.02299.

[5] Durieux, T., Martinez, M., Monperrus, M., & Wuttke, J. (2018). Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset. arXiv:1811.02429.

[6] J Jiang, , et al. (2017). A manual inspection of Defects4J bugs and its implications for automatic program repair . SCIS 2019.

[7] Catolino, G., Palomba, F., Zaidman, A., & Ferrucci, F. (2019). Not all bugs are the same: Understanding, characterizing, and classifying bug types. J. Systems and Software, 153, pp. 1–18.

[8] Andrade, R., Teixeira, C., Laranjeiro, N., & Vieira, M. (2025). An Empirical Study on the Classification of Bug Reports with Machine Learning. arXiv:2503.00660.

[9] Hirsch, T., & Hofer, B. (2022). Using textual bug reports to predict the fault category of software bugs. Array, 15, 100189.

[10] Colavito, G., Lanubile, F., Novielli, N., & Quaranta, L. (2024). Large Language Models for Issue Report Classification. Ital-IA 2024, CEUR Workshop Proceedings.

[11] Koyuncu, A. (2025). Exploring fine-grained bug report categorization with LLMs and prompt engineering. ACM TOSEM.

References (2/2)

[12] Chillarege, R., et al. (1992). Orthogonal Defect Classification — A Concept for In-Process Measurements. IEEE TSE, 18(11), pp. 943–956.

[13] Huang, B., & Ng, V., et al. (2011). AutoODC: Automated generation of Orthogonal Defect Classifications. ASE 2011.

[14] Thung, F., Lo, D., et al. (2012). Automatic Defect Categorization. WCRE 2012.

[15] Kang, S., Chen, B., Yoo, S., & Lou, J.-G. (2023). Explainable Automated Debugging via Large Language Model-driven Scientific Debugging. arXiv:2304.02195.

[16] Zeller, A. (2009). Why Programs Fail: A Guide to Systematic Debugging (2nd ed.). Morgan Kaufmann.

[17] Esfahani, et al. (2024). Understanding Defects in Generated Codes by Language Models. CASCON 2024. (arXiv:2408.13372)

[18] Mollik., et al. (2024). RCEGen: A Generative Approach for Automated Root Cause Analysis. MDPI Computers, 4(4), 29.

[19] Nong, Y., et al. (2024). Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities. arXiv:2402.17230.

[20] IBM. (2013). Orthogonal Defect Classification v 5.2 for Software Design and Code. IBM Research.

[21] Callaghan, D. (2024). Mining Bug Repositories for Multi-Fault Programs. defects4j-mf, GitHub.

Thank You